

SpringBoot

鸣谢：黑马程序员。（视频链接：【黑马程序员SSM框架教程_Spring+SpringMVC+Maven高级+SpringBoot+MyBatisPlus企业实用开发技术】https://www.bilibili.com/video/BV1Fi4y1S7ix?vd_source=b7f14ba5e783353d06a99352d23ebca9）

• SpringBoot

一、入门

1.入门案例

1.1 项目信息

1.2 开发步骤

1.3 最简SpringBoot程序包含的基础文件

1.4 Spring程序与SpringBoot程序对比

1.5 通过Spring官网创建SpringBoot工程

1.6 SpringBoot程序快速启动

2.简介

2.1 评价

2.2 起步依赖

2.3 切换Web服务器

二、基础配置

1.配置文件格式

1.1 修改服务器端口

P1 `application.properties`

P2 `application.yml`

P3 `application.yaml`

P4 配置文件优先级

2.YAML

2.1 概述

2.2 语法规则

2.3 YAML数据读取方式

P1 使用 `@Value` 读取单个数据

P2 封装全部数据到 `Environment` 对象

P3 创建自定义实体类封装指定数据

3.多环境启动

3.1 多环境启动配置

P1 YAML版

P2 `properties` 版

3.2 多环境启动 `cmd` 格式

3.3 Maven与SpringBoot多环境兼容

P1 Maven中设置多环境属性

P2 SpringBoot中引用Maven属性

P3 对资源文件开启对默认占位符的解析

4.配置文件分类

一、入门

📢 Important

SpringBoot是Pivotal团队开发的全新框架，旨在简化Spring应用的初始搭建及开发过程。

1.入门案例

1.1 项目信息

- 入门案例地址：[🔗springboot-quickstart](#)
- IntelliJ IDEA版本：2024.2.1

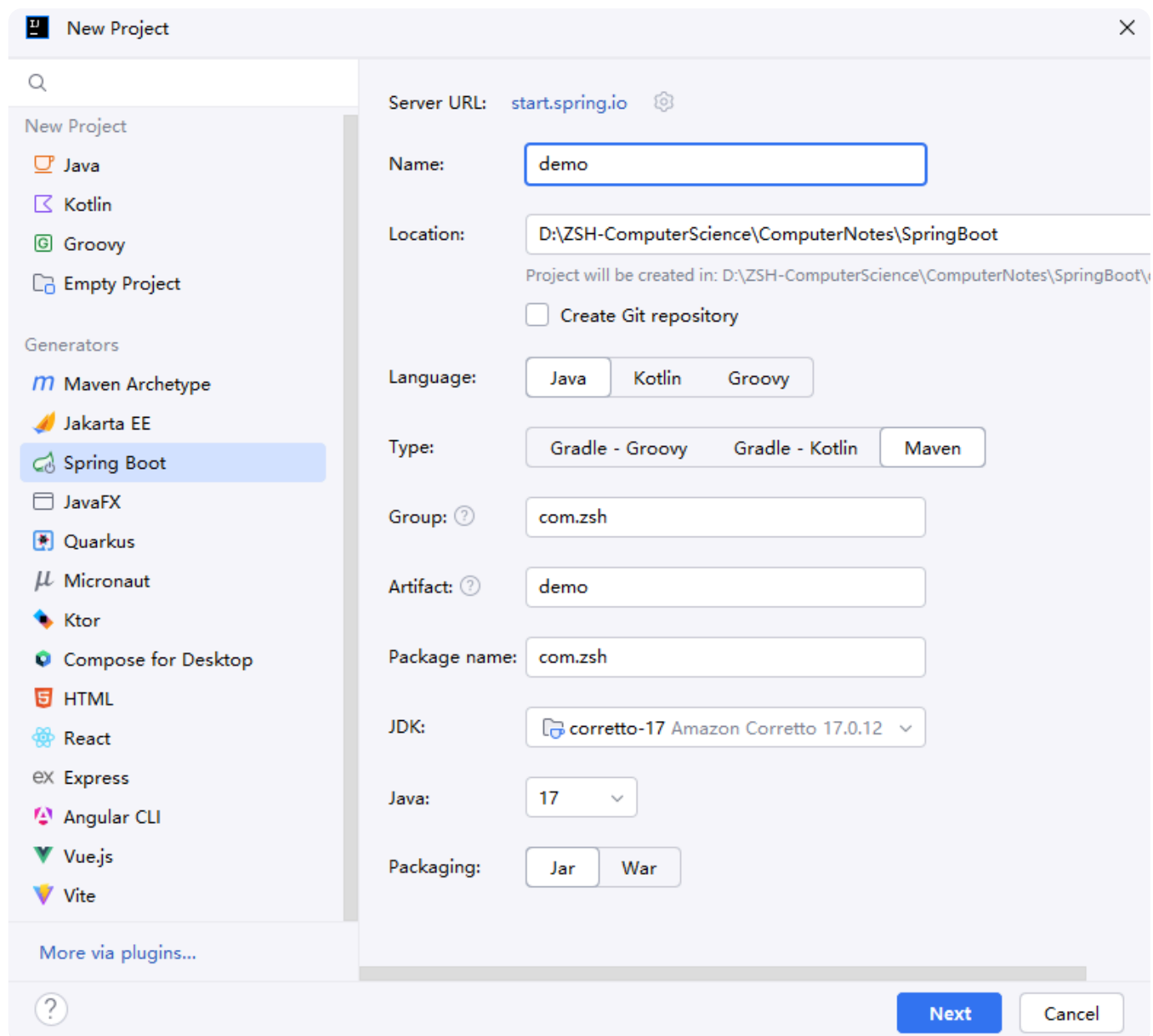
1.2 开发步骤

⚠ Caution

基于IntelliJ IDEA开发SpringBoot程序必须联网，才能加载到程序框架结构。

Step1: 创建新工程，在左侧栏目选择 **Spring Boot**，右侧指定工程相关基础信息：

- **Name** 和 **Location** 自定义。
- **Type** 要选择 **Maven**。
- **Package name** 修改为 **com.zsh**。
- **Packaging** 选择 **Jar** 即可。

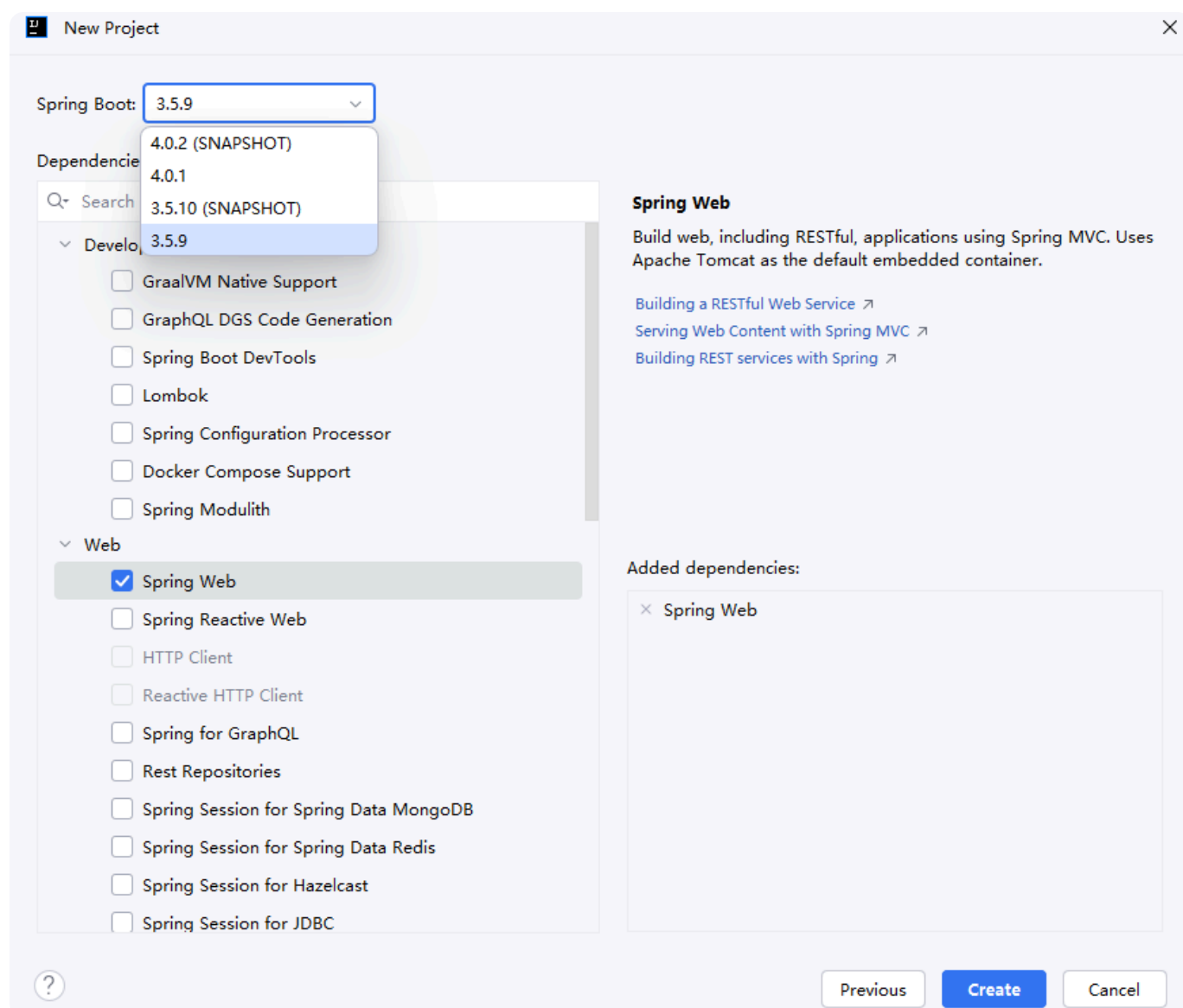


The image shows the 'New Project' dialog box in IntelliJ IDEA. On the left, under 'Generators', 'Spring Boot' is selected. The right panel shows the following configuration:

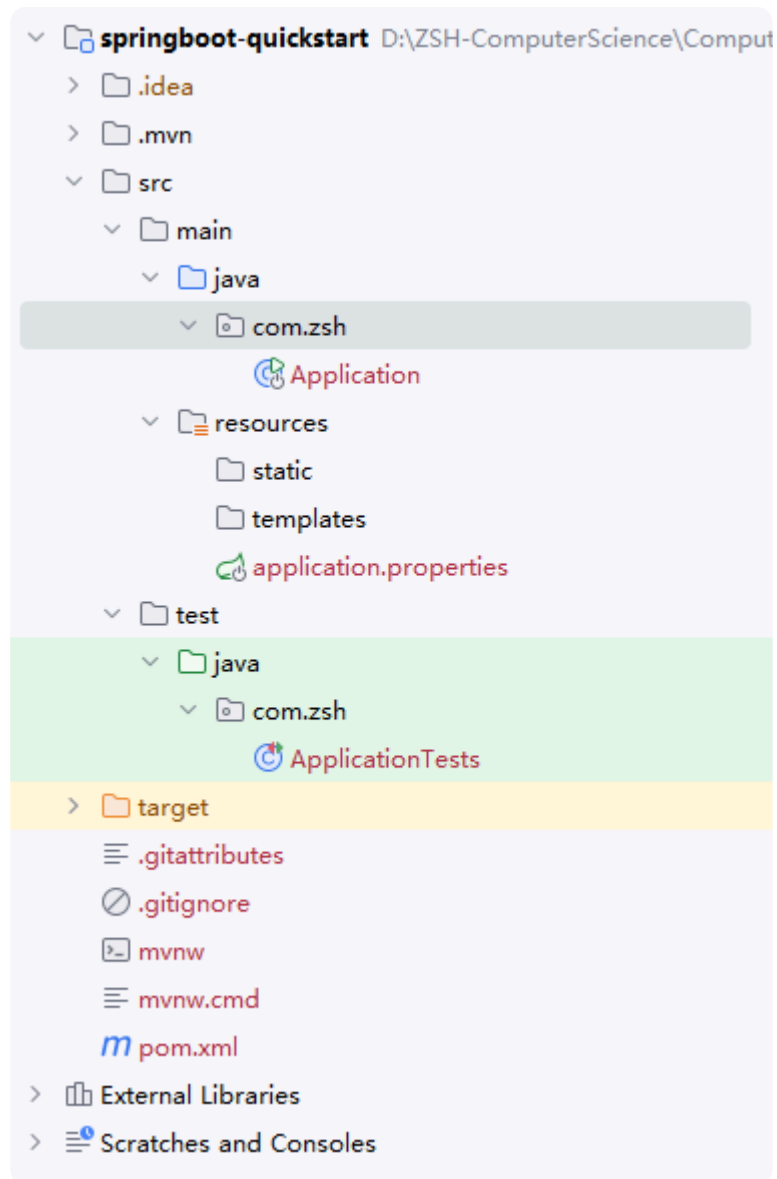
- Server URL: start.spring.io
- Name: demo
- Location: D:\ZSH-ComputerScience\ComputerNotes\SpringBoot
- Project will be created in: D:\ZSH-ComputerScience\ComputerNotes\SpringBoot\
- ☐ Create Git repository
- Language: Java (selected), Kotlin, Groovy
- Type: Gradle - Groovy, Gradle - Kotlin, Maven (selected)
- Group: com.zsh
- Artifact: demo
- Package name: com.zsh
- JDK: corretto-17 Amazon Corretto 17.0.12
- Java: 17
- Packaging: Jar (selected), War

At the bottom, there are 'Next' and 'Cancel' buttons.

Step2: 选择 **Spring Boot** 版本, 勾选项目需要的技术栈, 此处为 **Dependencies-Web-Spring Web**。



Step3: 点击 **Create** 创建工程, 可得如下项目结构。



Step4: 在 `src/main/java/com.zsh` 下新建 `controller` 包, 开发控制器类 `BookController`。

```
1 package com.zsh.controller;
2
3 import org.springframework.web.bind.annotation.*;
4
5 @RestController
6 @RequestMapping("/books")
7 public class BookController {
8
9     @GetMapping("/{id}")
10    public String getById(@PathVariable Integer id) {
11        System.out.println("id ==> " + id);
12        return "hello springboot";
13    }
14 }
```

```
1 | spring.application.name=springboot-quickstart
2 | server.port=8082
```

Step5: 启动系统自动生成的 `Application` 类，入门案例开发完毕。

1.3 最简SpringBoot程序包含的基础文件

- pom.xml :

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
  http://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>3.5.9</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>com.zsh</groupId>
```

```

12     <artifactId>springboot-quickstart</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14
15     <properties>
16         <java.version>17</java.version>
17     </properties>
18
19     <dependencies>
20         <dependency>
21             <groupId>org.springframework.boot</groupId>
22             <artifactId>spring-boot-starter-web</artifactId>
23         </dependency>
24         <dependency>
25             <groupId>org.springframework.boot</groupId>
26             <artifactId>spring-boot-starter-test</artifactId>
27             <scope>test</scope>
28         </dependency>
29     </dependencies>
30
31     <build>
32         <plugins>
33             <plugin>
34                 <groupId>org.springframework.boot</groupId>
35                 <artifactId>spring-boot-maven-plugin</artifactId>
36             </plugin>
37         </plugins>
38     </build>
39
40 </project>

```

- Application 类:

```

1 package com.zsh;
2
3 import org.springframework.boot.SpringApplication;
4 import
  org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 public class Application {
8
9     public static void main(String[] args) {
10         SpringApplication.run(Application.class, args);
11     }
12
13 }

```

1.4 Spring程序与SpringBoot程序对比

类/配置文件	Spring	SpringBoot
<code>pom.xml</code> 中的依赖坐标	手动添加	勾选添加
web3.0配置类 <code>ServletConfig</code>	手动开发	无
Spring/SpringMVC配置类	手动开发	无
控制器	手动开发	手动开发

1.5 通过Spring官网创建SpringBoot工程

- [Spring官网-创建SpringBoot工程](#)

spring initializr

Project

☐ Gradle - Groovy

☐ Gradle - Kotlin

☒ Maven

Language

☒ Java

☐ Kotlin

☐ Groovy

Spring Boot

☐ 4.0.2 (SNAPSHOT)

☐ 4.0.1

☐ 3.5.10 (SNAPSHOT)

☒ 3.5.9

Project Metadata

Group

com.zsh

Artifact

demo

Name

demo

Description

Demo project for Spring Boot

Package name

com.zsh

Packaging

☒ Jar

☐ War

Configuration

☒ Properties

☐ YAML

Java

☐ 25

☐ 21

☒ 17

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web

WEB

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

GENERATE CTRL + G

EXPLORE CTRL + SPACE

...

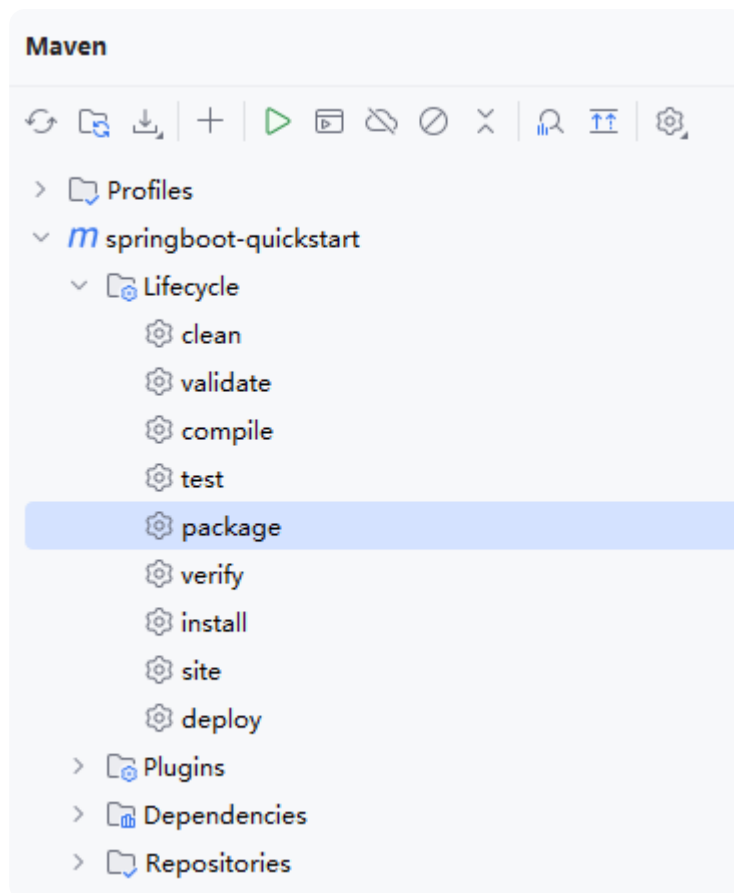
1.6 SpringBoot程序快速启动

⚠ Caution

jar包支持命令行启动需要依赖于maven插件的支持(`pom.xml`)，打包前请确认该插件是否存在：

```
1 <build>
2   <plugins>
3     <plugin>
4       <groupId>org.springframework.boot</groupId>
5       <artifactId>spring-boot-maven-plugin</artifactId>
6     </plugin>
7   </plugins>
8 </build>
```

Step1: 通过Maven构建指令 `package`（建议先 `clean` 一下），把SpringBoot项目打包成 `jar`。



Step2: 在项目 `jar` 包所处目录输入 `cmd` , 执行启动指令 `java -jar xxx.jar`。

```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.26200.7171]
(c) Microsoft Corporation. 保留所有权利。

D:\ZSH-ComputerScience\ComputerNotes\SpringBoot\springboot-quickstart\target>java -jar springboot-quickstart-0.0.1-SNAPSHOT.jar

:: Spring Boot ::                (v3.5.9)

2025-12-30T15:40:29.491+08:00 INFO 14476 --- [springboot-quickstart] [main] com.zsh.Application : Starting Application v0.0.1-SNAPSHOT using Java 21.0.6 with PID 14476 (D:\ZSH-ComputerScience\ComputerNotes\SpringBoot\springboot-quickstart\target\springboot-quickstart-0.0.1-SNAPSHOT.jar started by asus in D:\ZSH-ComputerScience\ComputerNotes\SpringBoot\springboot-quickstart\target)
2025-12-30T15:40:29.493+08:00 INFO 14476 --- [springboot-quickstart] [main] com.zsh.Application : No active profile set, falling back to 1 default profile: "default"
2025-12-30T15:40:30.272+08:00 INFO 14476 --- [springboot-quickstart] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8082 (http)
2025-12-30T15:40:30.286+08:00 INFO 14476 --- [springboot-quickstart] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-12-30T15:40:30.286+08:00 INFO 14476 --- [springboot-quickstart] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.50]
2025-12-30T15:40:30.316+08:00 INFO 14476 --- [springboot-quickstart] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2025-12-30T15:40:30.317+08:00 INFO 14476 --- [springboot-quickstart] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 776 ms
2025-12-30T15:40:30.650+08:00 INFO 14476 --- [springboot-quickstart] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8082 (http) with context path '/'
2025-12-30T15:40:30.668+08:00 INFO 14476 --- [springboot-quickstart] [main] com.zsh.Application : Started Application in 1.571 seconds (process running for 2.002)
```

2.简介

2.1 评价

- Spring程序的**缺点**:
 - 配置繁琐
 - 依赖设置繁琐
- SpringBoot程序的**优点**:
 - 自动配置
 - 起步依赖（简化依赖配置）
 - 辅助功能（内置 Tomcat 服务器，.....）

2.2 起步依赖

- `starter`：SpringBoot中常见依赖名称，定义了当前项目使用的所有依赖坐标，以达到**减少依赖配置**的目的。
- `parent`：所有SpringBoot项目要继承的项目，定义了若干个坐标版本号（依赖管理，而非依赖），以达到**减少依赖冲突**的目的。
- 实际开发中：使用任意坐标时，仅书写GAV中的GA, V由SpringBoot提供；如发生坐标错误，再指定V（要小心版本冲突）。

2.3 切换Web服务器

示例：切换到Jetty服务器，比Tomcat服务器更轻量级，可扩展性更强。

```
1 <dependencies>
2     <dependency>
3         <groupId>org.springframework.boot</groupId>
4         <artifactId>spring-boot-starter-web</artifactId>
5         <!-- 排除依赖: Tomcat -->
6         <exclusions>
7             <exclusion>
8                 <groupId>org.springframework.boot</groupId>
9                 <artifactId>spring-boot-starter-tomcat</artifactId>
10            </exclusion>
```

```
11         </exclusions>
12     </dependency>
13
14     <dependency>
15         <groupId>org.springframework.boot</groupId>
16         <artifactId>spring-boot-starter-jetty</artifactId>
17     </dependency>
18
19     <dependency>
20         <groupId>org.springframework.boot</groupId>
21         <artifactId>spring-boot-starter-test</artifactId>
22         <scope>test</scope>
23     </dependency>
24 </dependencies>
```

二、基础配置

1. 配置文件格式

1.1 修改服务器端口

💡 Tip

yml和yaml是同一个东西，没有本质区别——yml只是yaml的简写。

P1 application.properties

```
1 server.port=80
```

P2 application.yml

```
1 server:
2   port: 81 #port和81之间必须有空格
```

P3 application.yml

```
1 server:
2   port: 82 #port和82之间必须有空格
```

P4 配置文件优先级

当以上三种格式的配置文件同时存在时，properties>yaml>yml。

2.YAML

2.1 概述

- YAML(YAML Ain't Markup Language): 递归缩写，强调它不是标记语言，而是一种**人类可读的数据序列化格式**。
- 优点：
 - 可读性强
 - 容易与脚本语言交互
 - 以数据为核心，重数据轻格式。
- YAML文件扩展名：`.yml`（主流）、`.yaml`。

2.2 语法规则

- 大小写敏感。
- 属性层级关系使用多行描述，每行结尾使用冒号结束。
- 使用缩进表示层级关系，同层级左侧对齐，只允许使用空格（空格数无规定），**不允许使用Tab键**。
- 属性值前面必须添加空格，属性名与属性值之间使用 `:` 作为分隔。
- `#`表示注释。

示例如下：

```
1 enterprise:
2   name: zsh
3   age: 18
4   tel: 83225361
```

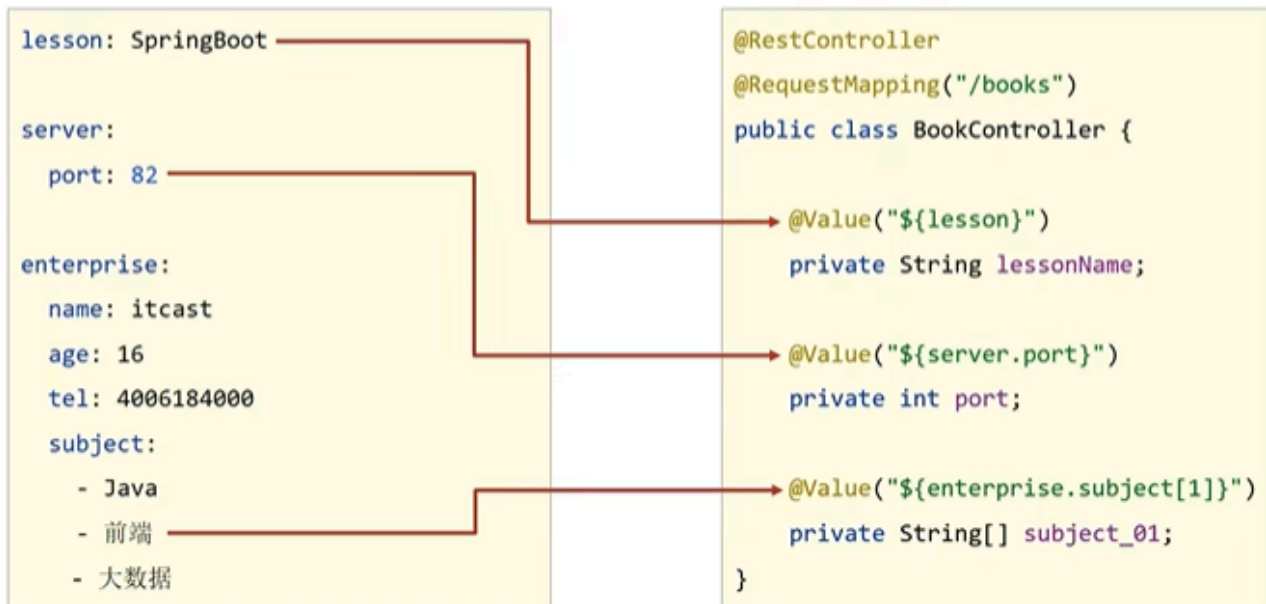
数组形式：

```
1 hobbies:
2   - sing
3   - dance
4   - rap
```

2.3 YAML数据读取方式

P1 使用 `@Value` 读取单个数据

- 使用@Value读取单个数据，属性名引用方式：\${一级属性名.二级属性名.....}



P2 封装全部数据到 `Environment` 对象

- 封装全部数据到 `Environment` 对象



P3 创建自定义实体类封装指定数据

- 自定义对象封装指定数据

```
lesson: SpringBoot
```

```
server:
```

```
  port: 82
```

```
enterprise:
```

```
  name: itcast
```

```
  age: 16
```

```
  tel: 4006184000
```

```
  subject:
```

```
    - Java
```

```
    - 前端
```

```
    - 大数据
```

```
@Component
```

```
@ConfigurationProperties(prefix = "enterprise")
```

```
public class Enterprise {
```

```
    private String name;
```

```
    private Integer age;
```

```
    private String[] subject;
```

```
}
```

```
@RestController
```

```
@RequestMapping("/books")
```

```
public class BookController {
```

```
    @Autowired
```

```
    private Enterprise enterprise;
```

```
}
```

自定义对象封装数据警告解决方案

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
```

3. 多环境启动

3.1 多环境启动配置

P1 YAML版

💡 Tip

用 `---` 来区分不同环境。

```
1 # 启动指定环境，此处是启动生产环境
2 spring:
3   profiles:
```



```
4         active: produce
5
6     ---
7
8     # 设置开发环境
9     spring:
10         config:
11             activate:
12                 on-profile: develop
13
14     # 开发环境具体参数设定
15     server:
16         port: 80
17
18     ---
19
20     # 设置生产环境
21     spring:
22         config:
23             activate:
24                 on-profile: produce
25
26     # 生产环境具体参数设定
27     server:
28         port: 81
29
30     ---
31
32     # 设置测试环境
33     spring:
34         config:
35             activate:
36                 on-profile: test
37
38     # 测试环境具体参数设定
39     server:
40         port: 82
```

P2 properties 版

- 主启动配置文件application.properties

```
spring.profiles.active=pro
```

- 环境分类配置文件application-**pro**.properties

```
server.port=80
```

- 环境分类配置文件application-**dev**.properties

```
server.port=81
```

- 环境分类配置文件application-**test**.properties

```
server.port=82
```

3.2 多环境启动 cmd 格式

```
1 java -jar xxx.jar --spring.profiles.active=环境名称
2 java -jar xxx.jar --server.port=测试人员指定端口
3 java -jar xxx.jar --spring.profiles.active=环境名称 --server.port=测试人员指定端口
```

3.3 Maven与SpringBoot多环境兼容

P1 Maven中设置多环境属性

```

<profiles>
  <profile>
    <id>dev_env</id>
    <properties>
      <profile.active>dev</profile.active>
    </properties>
    <activation>
      <activeByDefault>true</activeByDefault>
    </activation>
  </profile>
  <profile>
    <id>pro_env</id>
    <properties>
      <profile.active>pro</profile.active>
    </properties>
  </profile>
  <profile>
    <id>test_env</id>
    <properties>
      <profile.active>test</profile.active>
    </properties>
  </profile>
</profiles>

```

P2 SpringBoot中引用Maven属性

```

spring:
  profiles:
    active: ${profile.active}
---
spring:
  profiles: pro
server:
  port: 80
---
spring:
  profiles: dev
server:
  port: 81
---
spring:
  profiles: test
server:
  port: 82

```

P3 对资源文件开启对默认占位符的解析

```
<build>
  <plugins>
    <plugin>
      <artifactId>maven-resources-plugin</artifactId>
      <configuration>
        <encoding>utf-8</encoding>
        <useDefaultDelimiters>true</useDefaultDelimiters>
      </configuration>
    </plugin>
  </plugins>
</build>
```

4.配置文件分类

⚠ Caution

classpath 即类路径，就是当前项目及其子文件夹。

级别	名称	作用
1级（最高）	filesystem: config/application.yml	留作系统打包后设置通用属性。
2级	filesystem: application.yml	留作系统打包后设置通用属性。
3级	classpath: config/application.yml	用于系统开发阶段设置通用属性。
4级（最低）	classpath: application.yml	用于系统开发阶段设置通用属性。

